

Report – Esercizio Python

Obiettivo

L'attività è composta da due esercizi Python.

Nel primo esercizio è stata realizzata una funzione che riceve una lista di parole e restituisce una lista contenente la lunghezza di ogni parola.

Nel secondo esercizio, facoltativo, è stata realizzata una funzione per generare automaticamente password di diversa complessità in base alla scelta dell'utente.

Esercizio 1 – Calcolo della lunghezza delle parole

Obiettivo

L'obiettivo è creare una funzione che, data una lista A contenente diverse parole, restituisca una lista B di numeri interi. Ogni numero rappresenta la lunghezza della parola corrispondente presente nella lista A.

Passo 1 – Creazione della funzione

Per prima cosa è stata creata la funzione `lunghezza_parola`:

```
def lunghezza_parola(A):
```

```
    B = []
```

La funzione riceve come parametro A, cioè la lista contenente le parole.

All'interno della funzione viene creata una lista vuota chiamata B, nella quale verranno inseriti i risultati.

Passo 2 – Calcolo della lunghezza

Per analizzare ogni parola presente nella lista è stato utilizzato un ciclo `for`:

```
for parola in A:
```

```
    B.append(len(parola))
```

Il ciclo prende una parola alla volta dalla lista A. La funzione `len()` calcola il numero di caratteri presenti nella parola, mentre `append()` aggiunge il risultato alla lista B.

Passo 3 – Restituzione del risultato

Al termine del ciclo viene utilizzato:

`return B`

Il comando `return` restituisce la lista contenente le lunghezze delle parole.

Passo 4 – Inserimento dei dati

La lista utilizzata nel programma è:

`A = ['Cane', 'Gatto', 'elementare', 'mouse']`

Successivamente viene richiamata la funzione:

`B = lunghezza_parola(A)`

Il risultato viene infine visualizzato con:

`print(B)`

Il programma restituisce:

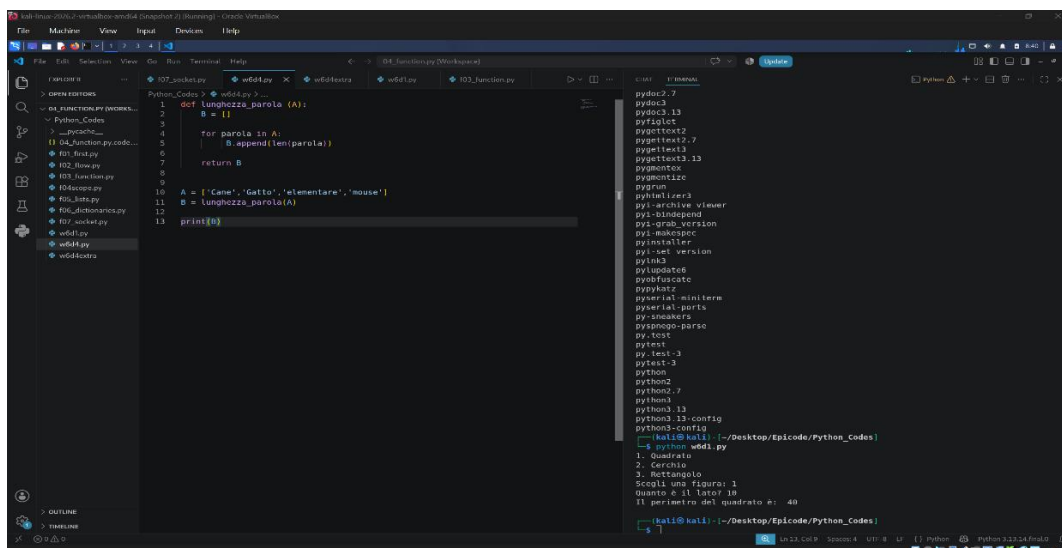
`[4, 5, 10, 5]`

Il risultato è corretto perché Cane contiene 4 caratteri, Gatto 5, elementare 10 e mouse 5.

Conclusioni dell'esercizio 1

L'esercizio ha permesso di utilizzare funzioni, liste, cicli `for`, `len()` e `append()`.

La funzione può essere utilizzata anche con altre liste di parole senza dover modificare il suo funzionamento.



The screenshot shows a Python IDE with a file explorer on the left, a code editor in the center, and a terminal on the right. The code editor contains the following Python code:

```
1 def lunghezza_parola(A):
2     B = []
3     for parola in A:
4         B.append(len(parola))
5     return B
6
7 A = ['Cane', 'Gatto', 'elementare', 'mouse']
8 B = lunghezza_parola(A)
9 print(B)
```

The terminal on the right shows the output of the program:

```
python3 wdd1.py
[4, 5, 10, 5]
```

Esercizio 2 – Generatore di password

Obiettivo

Il secondo esercizio, indicato nella traccia come facoltativo, consiste nella realizzazione di una funzione in grado di generare automaticamente una password.

Il programma permette all'utente di scegliere tra due tipi di password:

- **Semplice:** 8 caratteri alfanumerici.
- **Complessa:** 20 caratteri utilizzando i caratteri disponibili in `string.printable`.

Passo 1 – Importazione dei moduli

All'inizio del programma sono stati importati i moduli necessari:

```
import random
```

```
import string
```

Il modulo `random` viene utilizzato per scegliere casualmente i caratteri della password.

Il modulo `string` permette invece di utilizzare gruppi di caratteri già disponibili, come lettere, numeri e caratteri ASCII.

Passo 2 – Creazione della funzione

È stata creata la funzione:

```
def password_generator(tipo):
```

La funzione riceve come parametro `tipo`, che indica se l'utente desidera una password semplice oppure complessa.

Passo 3 – Password semplice

Se l'utente sceglie "semplice", viene eseguito:

```
if tipo == "semplice":
```

```
    caratteri = string.ascii_letters + string.digits
```

```
    lunghezza = 8
```

`string.ascii_letters` contiene le lettere dell'alfabeto, mentre `string.digits` contiene i numeri da 0 a 9.

I due gruppi vengono uniti nella variabile caratteri.

La lunghezza della password viene impostata a 8 caratteri.

Passo 4 – Password complessa

Se l'utente sceglie "complessa", viene utilizzato:

```
elif tipo == "complessa":
```

```
    caratteri = string.printable
```

```
    lunghezza = 20
```

In questo caso viene utilizzato string.printable, che contiene un insieme più ampio di caratteri ASCII, e la lunghezza della password viene impostata a 20 caratteri.

Passo 5 – Generazione della password

La password viene inizialmente impostata come stringa vuota:

```
password = ""
```

Successivamente viene utilizzato un ciclo:

```
for i in range(lunghezza):
```

```
    password += random.choice(caratteri)
```

Il ciclo viene ripetuto tante volte quanti sono i caratteri richiesti.

Ad ogni ripetizione, random.choice() sceglie casualmente un carattere dalla variabile caratteri, che viene aggiunto alla password.

Al termine, la funzione restituisce la password generata:

```
return password
```

Passo 6 – Scelta dell'utente

Il programma chiede all'utente quale tipo di password vuole generare:

```
scelta = input("Scegli il tipo di password se semplice o complessa: ")
```

Successivamente controlla che la scelta sia valida:

```
if scelta == "semplice" or scelta == "complessa":
```

```
    password = password_generator(scelta)
```

```
    print("La tua password è:", password)
```

L'esercizio mostra inoltre come una funzione possa essere utilizzata per automatizzare la generazione di password di diversa lunghezza e complessità.